LAB2 Частина 2. Оброблення даних про файли

Виконавець: **Потєхіна Валерія Ігорівна**, група **К-27**

Викладач: **Скибицький М.**

Нам знадобиться бібліотека `pandas` .

Бібліотека є сторонньою, отже потребує встановлення. Краще у віртуальне оточення.

Мала бути встановлена, якщо ні (клітина з імпортуванням виконується з помилкою), то треба встановити. Можна прямо з Jupyter. Тільки не під час пари.

Наступну клітину можна розкоментовувати та виконувати тільки, якщо в якості ядра (kernel) вибране віртуальне оточення!

In [1]:
```
#!pip install pandas
```

Якщо встановлювати не в оточення, то хоча б тільки для користувача (з ключем --user).

In [2]:
```
#!pip3.14 install pandas --user    # Linux
```

In [3]:
```
#!py -3.14 -m pip install pandas --user    # Windows
```

In [4]:
```
from pathlib import Path
import pandas as pd
print(f'pandas {pd.__version__}')
```

```
pandas 2.3.2
```

Постановка задачі

Згадка про минуле (Лабораторну роботу №1)

Протягом навчання програмні проєкти з різних предметів зберігалися в папках з переліку `folders` , який оброблявся в лабораторній роботі №1.

Щоб профільтрувати допоміжні файли середовищ, було введено "білий список" розширень (`WHITE_LIST`) та список розширень файлів з текстом програм(задається) (`CODE_LIST`).

Також було написано функцію `kind` , що за розширенням повертає рядок з символом C,W або O відповідно для файлів з текстом програм (відповідно до переліку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список).

У результаті виконання лабораторної роботи для кожного місяця та кожного розширення, що зустрічаються при перегляді файлів, розміщених у папках зі списку, було знайдено таку інформацію:

рік, місяць, розширення, кількість файлів, сумарний обсяг у байтах, кількість їх рядків.

Кількість рядків наводиться тільки для розширень категорії C; білі та порожні рядки не рахуються; для інших розширень ця клітина таблиці залишається незаповненою.

За цією інформацією було побудовано csv-файл з іменем `lab1_1.csv` (кодування тільки utf-8), на початку якого розміщено рядок з заголовками: year, mon, ext, cnt, size, lines.

In [5]:
```
CODE_LIST = {'.cpp', '.hpp', '.c', '.h', '.py', '.pyw', '.cs', '.js',
'.java'}
WHITE_LIST = CODE_LIST | {'.html', '.json', '.txt', '.ipynb'}
```

In [6]:
```
headers = ['year', 'mon', 'ext', 'cnt', 'size', 'lines']
```

За інформацією з csv-файлу з'ясувати

1. Скільки різних розширень зустрічається у папках з заданого списку, скільки файлів з кожним розширенням.
   (Для цієї задачі розглядаємо всі розширення, а не тільки "білого списку".)

   Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

2. За датами створення файлів для кожного місяця, починаючи з вересня 2024-го року, знайти таку інформацію:
   кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.

   Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Зверніть увагу на підписи стовпчиків, що будуть виводитись.

Всі значення округлити та привести до цілого типу.

Завантаження даних

Задаємо шлях до файлу `lab1_1.csv` .

In [7]:
```
path = Path("lab1_1.csv")
```

Завантажимо дані за допомогою функції `pd.read_csv` (документація тут).

Зверніть увагу на те, що:

1. файл записано в кодуванні `utf-8`
2. перший рядок файлу містить заголовки year, mon, ext, cnt, size, lines
3. значення у стовпчику `lines` можуть бути відсутні

▸ Підказки.

▸ Ще більше підказок

In [8]:
```
data = pd.read_csv(path, encoding='utf-8', header=0, sep=",", dtype=
{"lines": "Int64"})
data.head(3)
```

Out[8]:

	year	mon	ext	cnt	size	lines
0	2023	2	.h	1	3953	125
1	2023	3	.h	1	978597	30637
2	2024	2	.h	1	737139	9056

In [9]:
```
data.tail(3)
```

Out[9]:

	year	mon	ext	cnt	size	lines
12	2026	2	.js	2	66967	1653
13	2026	2	.py	3	45995	1125
14	2026	2	.txt	3	13289	<NA>

Подивимось інформацію про завантажені дані.
Документація на метод `info`.

In [10]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    15 non-null     int64
 1   mon     15 non-null     int64
 2   ext     15 non-null     object
 3   cnt     15 non-null     int64
 4   size    15 non-null     int64
 5   lines   12 non-null     Int64
dtypes: Int64(1), int64(4), object(1)
memory usage: 747.0+ bytes
```

Бачимо, що відсутні значення є не тільки в стовпчику `lines`, що природньо, бо кількість рядків обчислювалась не для всіх розширень, але і в стовпчику `ext` з розширеннями.

Замінимо відсутні розширення порожнім рядком.

Використаємо метод `fillna`.

Слід замінити тільки відсутні розширення, тому значення параметра `value` слід зробити словником, що для кожного стовпчика, який слід оброблювати, задає значення-заповнювач.

Не забудьте, що перетворення слід виконувати "на місці".

▸ Підказка (тільки за повного безвихіддя)

Альтернатива. Можна спробувати перетворити тільки стовпчик `data.ext`.
▸ Підказка (тільки за повного безвихіддя)

Перший спосіб більш економний за пам'яттю. Другий спосіб корисний тоді, коли для різних стовпчиків використовують різні алгоритми заповнення відсутніх значень.

In [11]:
```
data.fillna(value={'ext': ''}, inplace=True)
data.head(3)
```

Out[11]:

	year	mon	ext	cnt	size	lines
0	2023	2	.h	1	3953	125
1	2023	3	.h	1	978597	30637
2	2024	2	.h	1	737139	9056

In [12]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   year    15 non-null     int64
 1   mon     15 non-null     int64
 2   ext     15 non-null     object
 3   cnt     15 non-null     int64
 4   size    15 non-null     int64
 5   lines   12 non-null     Int64
dtypes: Int64(1), int64(4), object(1)
memory usage: 747.0+ bytes
```

Тепер незаповнені дані залишились тільки у стовпчику `lines`.

Ні в якому разі там не повинно бути нулів!

Групування та групові операції

In [13]:
```
data.head()
```

Out[13]:

	year	mon	ext	cnt	size	lines
0	2023	2	.h	1	3953	125
1	2023	3	.h	1	978597	30637
2	2024	2	.h	1	737139	9056
3	2024	10	.cpp	1	1215	41
4	2024	11	.cpp	1	5248	263

Задача 1
Скільки різних розширень зустрічається в папках з заданого списку, скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Скільки різних розширень зустрічається в папках з заданого списку?

Найпростіше: взяти стовпчик таблиці, що зберігає розширення, та знайти кількість унікальних значень.

▸ Підказки

In [14]:
```
data.ext.nunique()
```

Out[14]: 6

Скільки файлів з кожним розширенням?

Відповідну інформацію вивести на екран у табличному вигляді. Рядки таблиці містять розширення, кількість файлів та ознаку (C,W,O відповідно для файлів з текстом програм (розширення з списку `CODE_LIST`), інших з білого списку, та тих, що не потрапили в білий список). Інформація подається за спаданням кількості файлів.

Спочатку треба підсумувати інформацію по кожному розширенню. Допоможе метод `groupby` документація тут.

Групувати слід за розширенням (параметр `by`) і для кожної групи обчислити суму кількостей файлів (метод `sum`).

Щоб не сумувати роки, місяці тощо, групування слід застосувати до вибірки потрібних стовпчиків. Маємо вибрати стовпчик, за яким групуємо, і стовпчик, який сумуємо.

▸ Підказка

Результат присвоїти змінній `task1`.

In [15]:
```
task1 = data[["ext", "cnt"]].groupby(by=["ext"]).sum()
task1
```

Out[15]:

	cnt
ext	
.cpp	7
.h	5
.ipynb	1
.js	2
.py	3
.txt	4

Відсортувати за спаданням кількості файлів. Допоможе метод `sort_values` (документація тут).

Не забудьте вказати, що сортування відбувається за спаданням та "на місці".

▸ Підказки

In [16]:
```
task1.sort_values(by='cnt', ascending=False, inplace=True)
task1.head(3)
```

Out[16]:

	cnt
ext	
.cpp	7
.h	5
.txt	4

За умовою ще треба додати стовпчик з ознакою категорії розширення файлу W,C,O (див. умову).
У лабораторній роботі №1 була написана функція `kind`, що за розширенням повертає рядок з шуканою ознакою. Використаємо її.

In [17]:
```
def kind(extension):
    if extension in CODE_LIST:
        return 'C'
    elif extension in WHITE_LIST:
        return 'W'
    else:
        return 'O'
```

Додамо до таблиці стовпчик з ознаками розширень файлів.

Зараз розширення є індексом. За допомогою методу метод `reset_index` (документація тут)
зробимо його стовпчиком таблиці.

Не забувайте виконувати перетворення "на місці".

In [18]:
```
task1.reset_index(inplace=True)
task1.head(3)
```

Out[18]:

	ext	cnt
0	.cpp	7
1	.h	5
2	.txt	4

Тепер вже додаємо стовпчик з ознаками.

▸ Підказки

In [19]:
```
task1['kind'] = task1.ext.apply(kind)
task1.head(3)
```

Out[19]:

	ext	cnt	kind
0	.cpp	7	C
1	.h	5	C
2	.txt	4	W

За допомогою методу `set_index` (документація тут) знов зробимо розширення індексом таблиці.

Тут: Значенням параметра `keys` має стати ім'я стовпчика, що слід використати як індекс.

Не забувайте виконувати перетворення "на місці".

In [20]:
```
task1.set_index(keys='ext', inplace=True)
task1.head(3)
```

Out[20]:

	cnt	kind
ext		
.cpp	7	C
.h	5	C
.txt	4	W

Змінимо написи.

In [21]:
```
task1.index.name='Розширення'
task1.columns=['Кількість файлів','Ознака']
task1.head()
```

Out[21]:

	Кількість файлів	Ознака
Розширення		
.cpp	7	C
.h	5	C
.txt	4	W
.py	3	C
.js	2	C

In [22]:
```
task1.head(20)
```

Out[22]:

	Кількість файлів	Ознака
Розширення		
.cpp	7	C
.h	5	C
.txt	4	W
.py	3	C
.js	2	C
.ipynb	1	W

Задача 2 За датами створення файлів для кожного місяця знайти таку інформацію:
кількість файлів з розширеннями з "білого списку", їх сумарний розмір у кілобайтах, кількість їх рядків (враховувати тільки для файлів категорії C), середній розмір у байтах, середню кількість рядків.

Відповідну інформацію вивести на екран у табличному вигляді в порядку, оберненому до хронологічного.

Всі значення округлити та привести до цілого типу.

План:

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.

2. Вибрати дані, що стосуються розширень з "білого списку".

3. Згрупувати вибрані дані в межах року та місяця.

4. За згрупованою інформацією обчислити потрібні агрегати.

5. Виконати перетворення стовпчиків та відсортувати.

Щоб за невдачі повторно не завантажувати дані, створимо їх копію.

In [23]:
```
data1 = data.copy()
```

1. Додати до початкової таблиці `data` стовпчик `kind`, що містить ознаки розширень.

Аналогічно тому, що виконувалось раніше, додати до таблиці `data1` стовпчик `kind`, що містить ознаки.

In [24]:
```
data1['kind'] = data1.ext.apply(kind)
data1.head(3)
```

Out[24]:

	year	mon	ext	cnt	size	lines	kind
0	2023	2	.h	1	3953	125	C
1	2023	3	.h	1	978597	30637	C
2	2024	2	.h	1	737139	9056	C

2. Вибрати дані, що стосуються розширень з "білого списку".

Запишіть вираз, що задає вибірку тільки інформації про розширення з "білого списку".
Щоб далі не копіювати сам вираз, присвойте його значення змінній `selection`. (Це є зручно, коли експериментуємо, у продакшн потреби в цій змінній насправді не буде, замість неї можна буде використовувати вираз безпосередньо.)

▸ Підказка

In [25]:
```
selection = data1[data1.kind != 'O']
selection = selection.query("(year == 2024 and mon >= 9) or (year > 2024)")
selection.head()
```

Out[25]:

	year	mon	ext	cnt	size	lines	kind
3	2024	10	.cpp	1	1215	41	C
4	2024	11	.cpp	1	5248	263	C
5	2025	2	.cpp	1	2733	116	C
6	2025	3	.cpp	2	6257	278	C
7	2025	3	.h	2	7047	177	C

3. Згрупувати вибрані дані за ознаками в межах року та місяця.

Допоможе метод `groupby` документація тут.

Групувати слід за роком та місяцем (параметр `by` має бути списком). Оскільки далі будемо обчислювати статистики для розміру та кількості рядків, то перед групуванням з таблиці слід вибрати стовпчики, за якими групуємо і за якими обчислюємо статистики.

In [26]:
```
task2 = selection[['year', 'mon', 'cnt', 'size', 'lines']].groupby(by=['year', 'mon'])
```

Далі необхідно знайти кількість файлів за кожен місяць, їх сумарний та середній розмір, а також сумарну та середню кількість рядків.

У результаті групування отримався об'єкт `DataFrameGroupBy` (документація тут).

До нього застосовний метод `agg`, що обчислює статистики (агрегатні характеристики) (документація тут).

Для стовпчика `cnt` будемо шукати суму значень (`sum`), для стовпчика `size` будемо шукати суму значень та середнє значення (`mean`), для стовпчика `lines` будемо шукати суму значень та середнє значення.

Першим параметром методу `agg` зробимо словник, що задає які статистики для яких стовпчиків слід обчислювати. Ключами словника мають бути імена стовпчиків (рядки), а значеннями списки статистик або окремі статистики (статистики також задаємо рядками-іменами)).

Результат агрегування зробимо значенням змінної `res2`.

In [27]:
```
res2 = task2.agg({'cnt': 'sum', 'size': ['sum', 'mean'], 'lines':
['sum', 'mean']})
res2.head()
```

Out[27]:

		cnt	size		lines	
		sum	sum	mean	sum	mean
year	mon					
2024	10	1	1215	1215.0	41	41.0
	11	1	5248	5248.0	263	263.0
2025	2	1	2733	2733.0	116	116.0
	3	4	13304	6652.0	455	227.5
	5	1	1022	1022.0	48	48.0

Оскільки відбувалось групування по кільком стовпчикам та для окремих стовпчиків обчислювалось кілька характеристик, то отримали складені індекси (multindex) і рядків, і стовпчиків.

Поточний індекс стовпчиків.

In [28]:
```
res2.columns
```

Out[28]:
```
MultiIndex([(  'cnt',  'sum'),
            ( 'size',  'sum'),
            ( 'size', 'mean'),
            ('lines',  'sum'),
            ('lines', 'mean')],
           )
```

Перейдемо до простого індексу стовпчиків.

In [29]:
```
res2.columns = res2.columns.to_flat_index()
res2.columns
```

Out[29]:
```
Index([   ('cnt', 'sum'),   ('size', 'sum'),  ('size', 'mean'),
        ('lines', 'sum'), ('lines', 'mean')],
       dtype='object')
```

In [30]:
```
res2.head(3)
```

Out[30]:

		(cnt, sum)	(size, sum)	(size, mean)	(lines, sum)	(lines, mean)
year	**mon**					
2024	**10**	1	1215	1215.0	41	41.0
	11	1	5248	5248.0	263	263.0
2025	**2**	1	2733	2733.0	116	116.0

Змінимо назви стовпчиків.

In [31]:
```
res2.columns=['Кількість', 'Сумарний розмір', 'Середній розмір',
'Кількість рядків', 'Середня кількість рядків',]
res2.head(3)
```

Out[31]:

		Кількість	Сумарний розмір	Середній розмір	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**10**	1	1215	1215.0	41	41.0
	11	1	5248	5248.0	263	263.0
2025	**2**	1	2733	2733.0	116	116.0

Виразимо сумарний розмір у kb.

Для цього слід поділити значення на 1024, округлити. Додатково перетворимо на цілий тип (метод `astype` дозволяє виконувати перетворення типів, документація тут). Також не забудемо змінити підпис стовпчика.

Переходимо до кілобайтів.

In [32]:
```
res2['Сумарний розмір'] = (res2['Сумарний розмір'] /
1024).round().astype(int)
```

Змінюємо підпис стовпчиків

In [33]:
```
res2.columns = ['Кількість', 'Сумарний розмір, kb', 'Середній розмір,
b', 'Кількість рядків', 'Середня кількість рядків']
```

In [34]:
```
res2.head(3)
```

Out[34]:

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**10**	1	1	1215.0	41	41.0
	11	1	5	5248.0	263	263.0
2025	**2**	1	3	2733.0	116	116.0

Округлити середній розмір та середню кількість рядків, перейти до цілих значень.

In [35]:
```
res2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
MultiIndex: 7 entries, (np.int64(2024), np.int64(10)) to
(np.int64(2026), np.int64(2))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 7 non-null      int64
 1   Сумарний розмір, kb       7 non-null      int32
 2   Середній розмір, b        7 non-null      float64
 3   Кількість рядків          7 non-null      Int64
 4   Середня кількість рядків  7 non-null      Float64
dtypes: Float64(1), Int64(1), float64(1), int32(1), int64(1)
memory usage: 393.0 bytes
```

Ось тільки тепер замінимо відсутні значення кількості рядків нулями!

Якщо б це було зроблено раніше, то середня кількість рядків була б обрахована неправильно (на неї вплинули б нульові значення замість відсутніх).

▸ Підказка

In [36]:
```
res2.fillna(0, inplace=True)
res2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
MultiIndex: 7 entries, (np.int64(2024), np.int64(10)) to
(np.int64(2026), np.int64(2))
Data columns (total 5 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Кількість                 7 non-null      int64
 1   Сумарний розмір, kb       7 non-null      int32
 2   Середній розмір, b        7 non-null      float64
 3   Кількість рядків          7 non-null      Int64
 4   Середня кількість рядків  7 non-null      Float64
dtypes: Float64(1), Int64(1), float64(1), int32(1), int64(1)
memory usage: 393.0 bytes
```

Тепер можна перетворювати типи стовпчиків.

In [37]:
```
fname = 'Середній розмір, b'
res2[fname] = res2[fname].round().astype(int)

fname = 'Середня кількість рядків'
res2[fname] = res2[fname].round().astype(int)
```

In [38]:
```
res2
```

Out[38]:

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2024	**10**	1	1	1215	41	41
	11	1	5	5248	263	263
2025	**2**	1	3	2733	116	116
	3	4	13	6652	455	228
	5	1	1	1022	48	48
	10	2	3	1478	62	62
2026	**2**	9	124	31795	2778	1389

Відсортуємо в порядку, оберненому до хронологічного.

У даному випадку слід сортувати індекс.
Допоможе метод `sort_index` (документація тут).

In [39]:
```
res2.sort_index(ascending=[False, False], inplace=True)
res2
```

Out[39]:

		Кількість	Сумарний розмір, kb	Середній розмір, b	Кількість рядків	Середня кількість рядків
year	**mon**					
2026	**2**	9	124	31795	2778	1389
2025	**10**	2	3	1478	62	62
	5	1	1	1022	48	48
	3	4	13	6652	455	228
	2	1	3	2733	116	116
2024	**11**	1	5	5248	263	263
	10	1	1	1215	41	41

Частину 2 зроблено!

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.